

研发系统监控告警使用手册

研发系统监控告警使用手册	1
概览	3
应用相关：	3
Java 相关：	4
操作系统相关：	4
简介	5
详细配置步骤	5
新建 folder：	5
新建 dashboard：	6
新建监控项:	7
添加用户	10
分配权限	11
可配置的部分监控项目	12
应用相关：	13
Java 相关	14
操作系统相关	15

概览

可配置监控应用相关、Java 相关以及操作系统基础资源相关监控项；

配置结果如下：

http://172.16.4.144:3000/d/Op5LVq1mz111/carloan_java

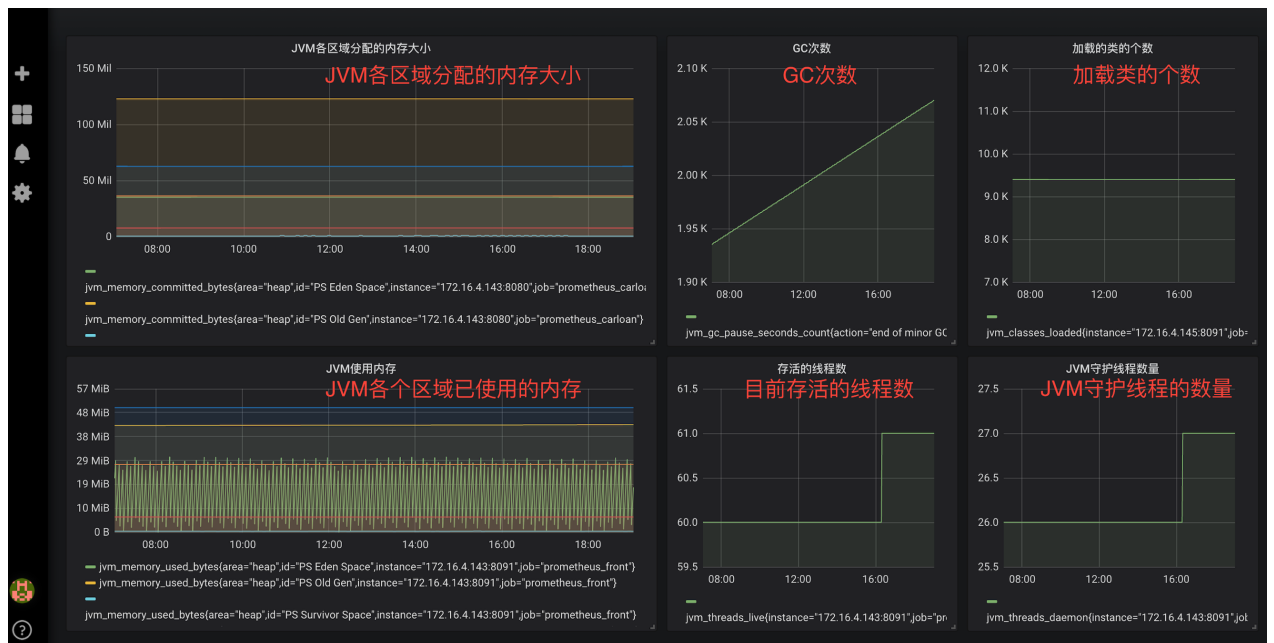
账号密码：carloan carloan

应用相关：

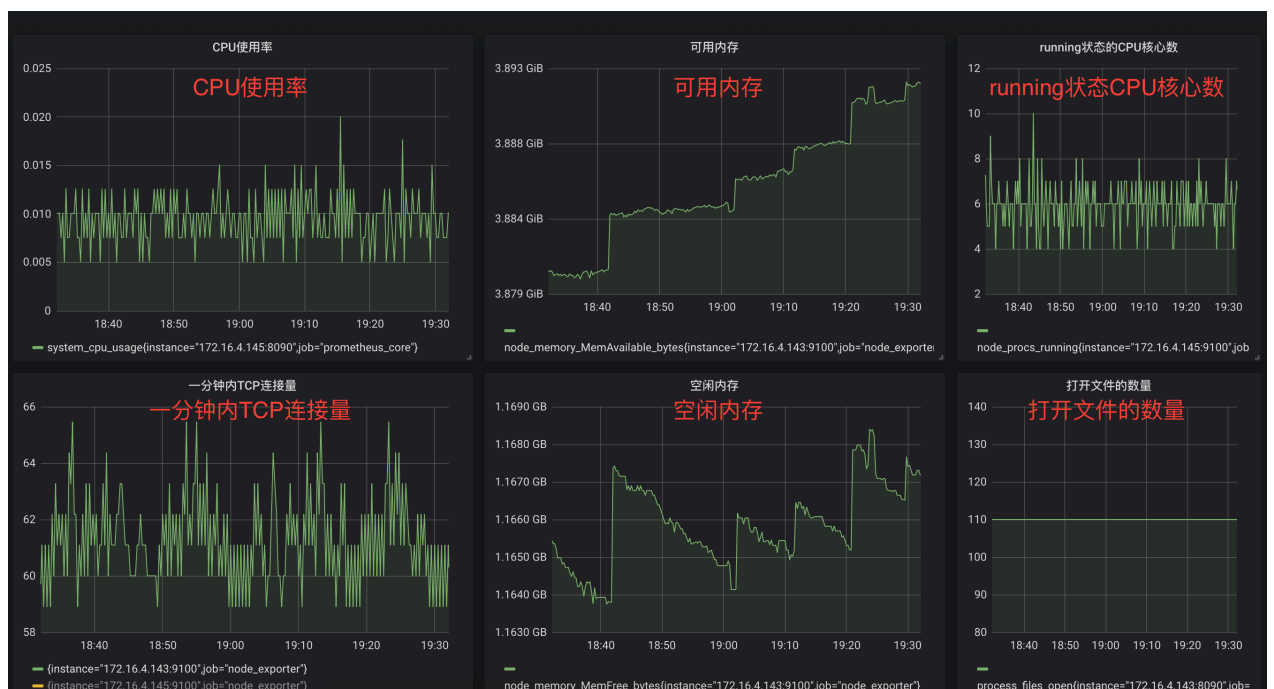


可自定义监控 http 状态不为 200、接口方法超时等监控项。

Java 相关：



操作系统相关：



简介

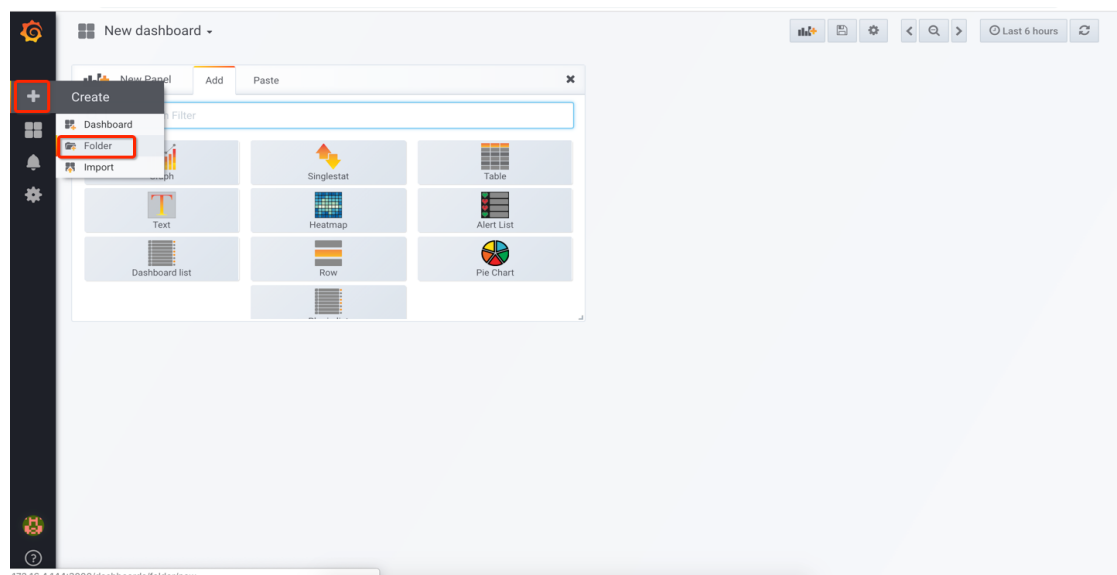
通过相关配置，可监控 JVM 加载类的数量、JVM 各个区域占用的空间、GC 回收的内存大小、GC 耗时、JVM 中存活的线程数量、守护线程数量等等 java 环境相关监控项；Tomcat 线程相关、Tomcat session 相关信息、应用 http 接口的请求耗时、http 接口请求次数等等应用相关监控项；系统 CPU 使用率、内存以及网络相关基础监控信息。

详细配置步骤

可按照不同的应用建立不同的 folder（文件夹），在 folder 中建立不同 dashboard 来监控不同类型的监控项。最后在 dashboard 中单独添加监控项。

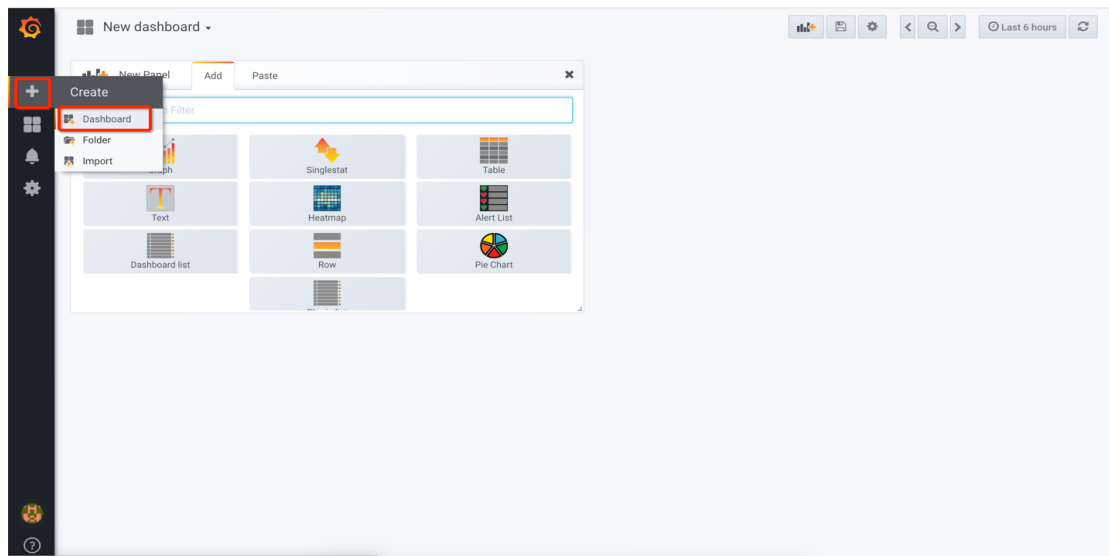
登录配置好的 Grafana。

新建 folder：

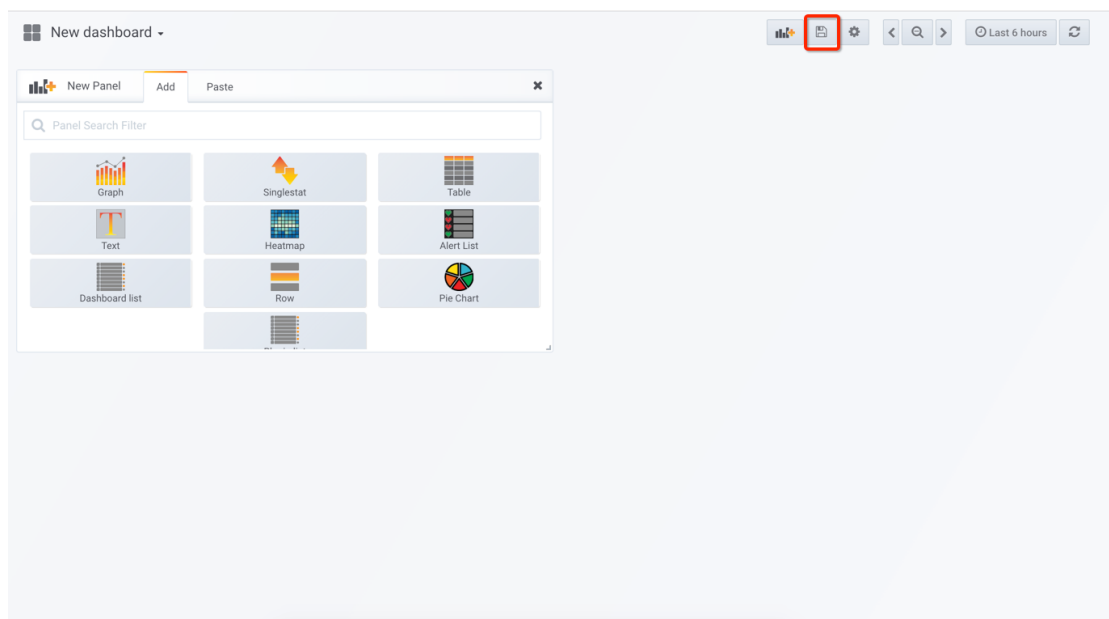


输入 name 点击 create 即可。

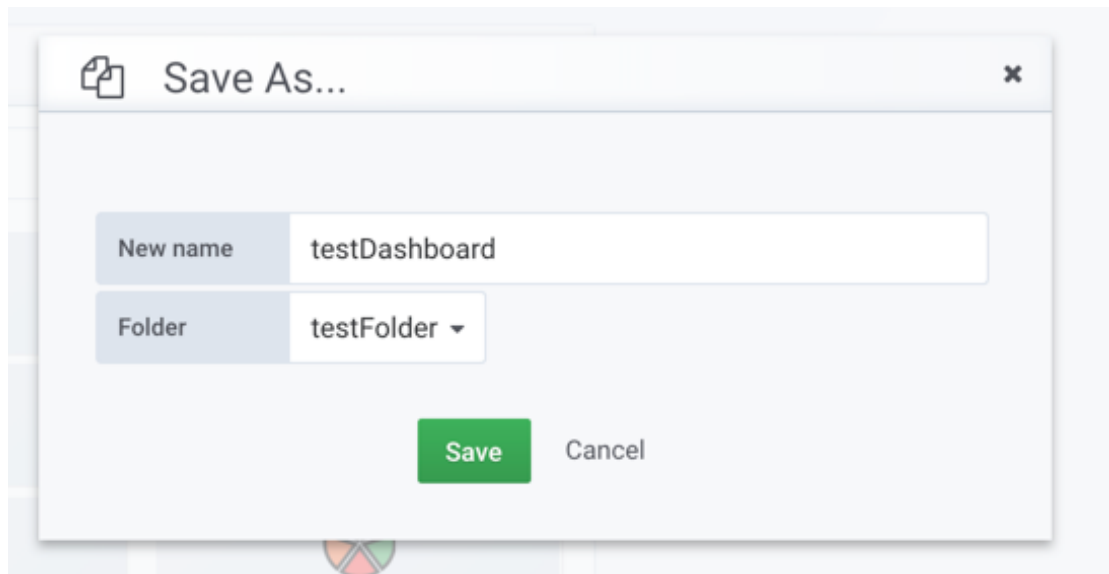
新建 dashboard：



点击保存：

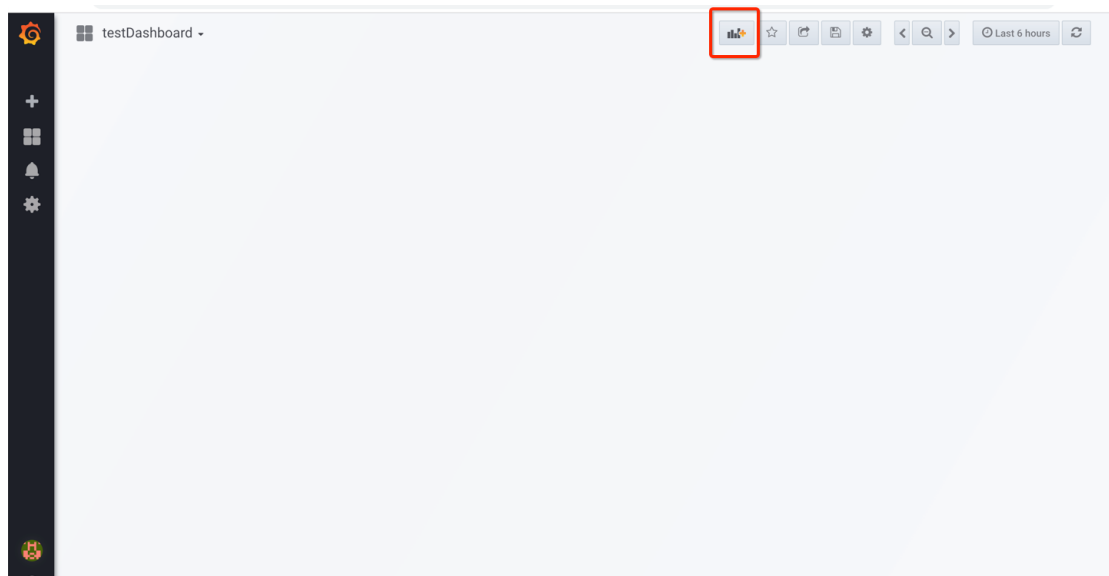


填写 name 选择上一步建立的 folder

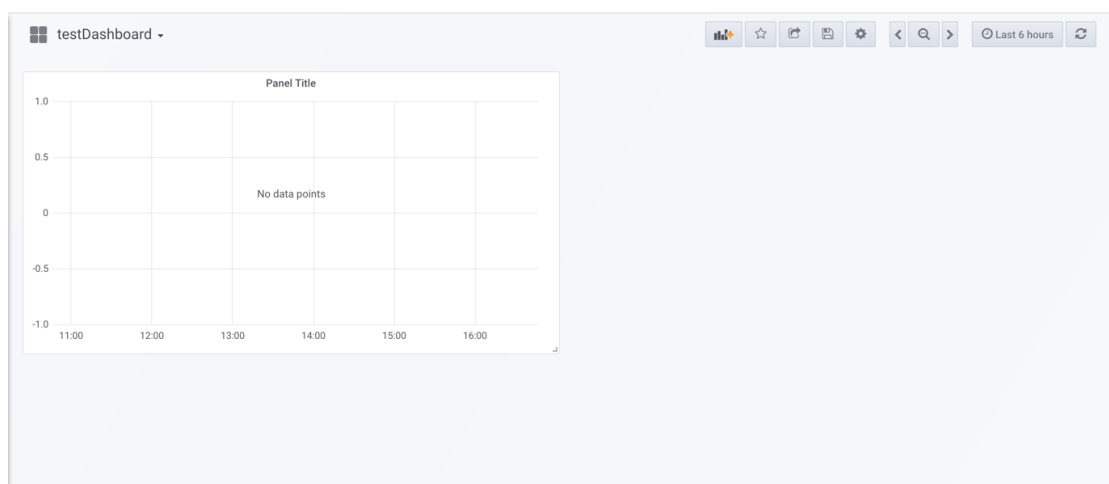
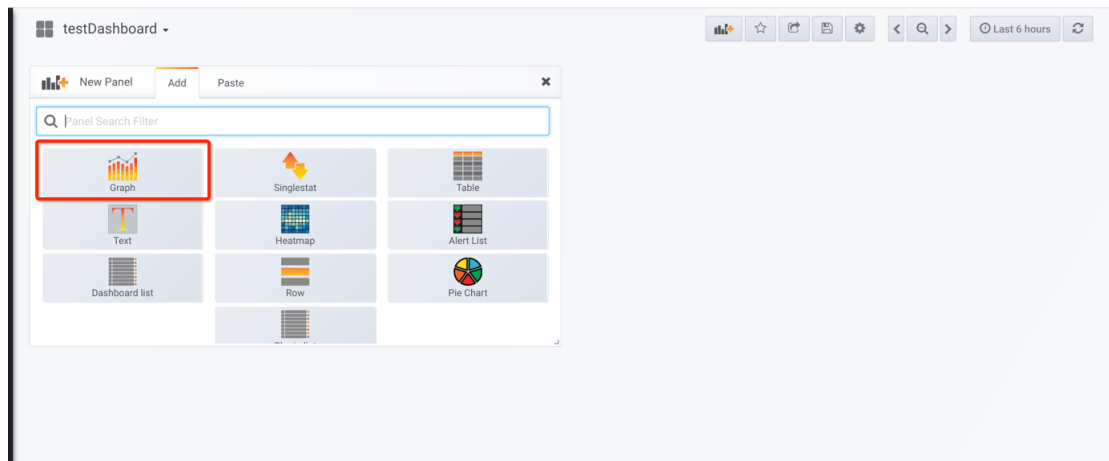


新建监控项:

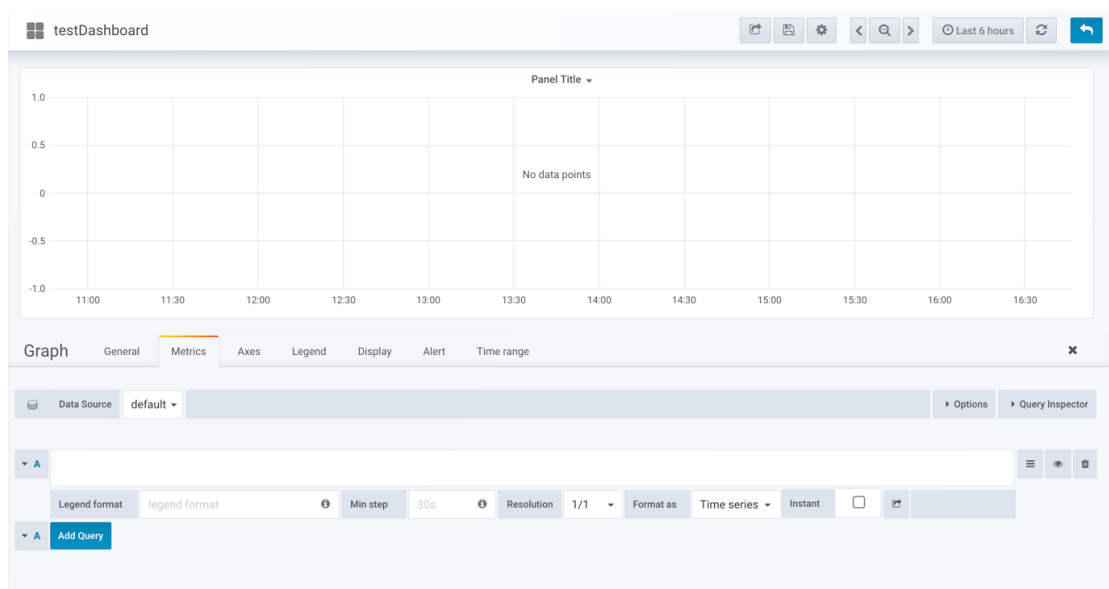
在上一步新建的 dashboard 里，点击 add panel



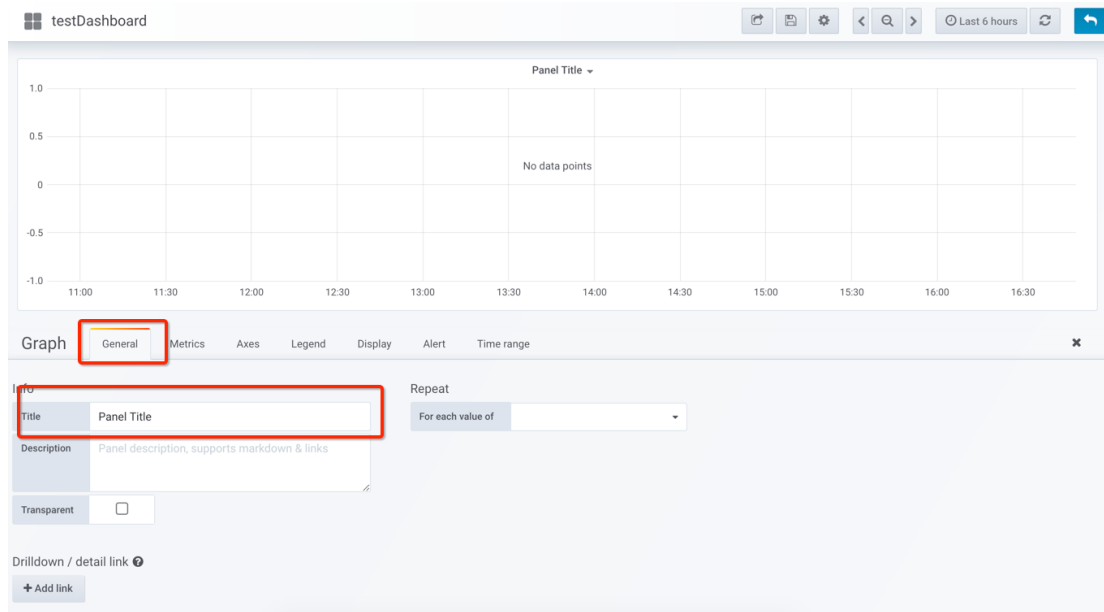
点击 graph :



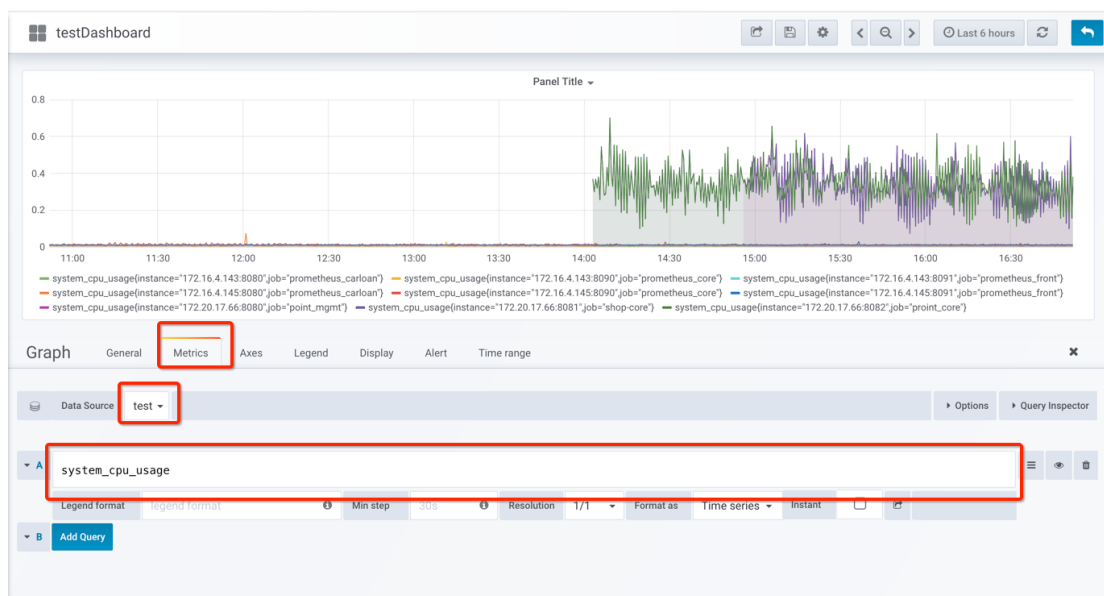
配置这个 panel 点击 panel title -> edit



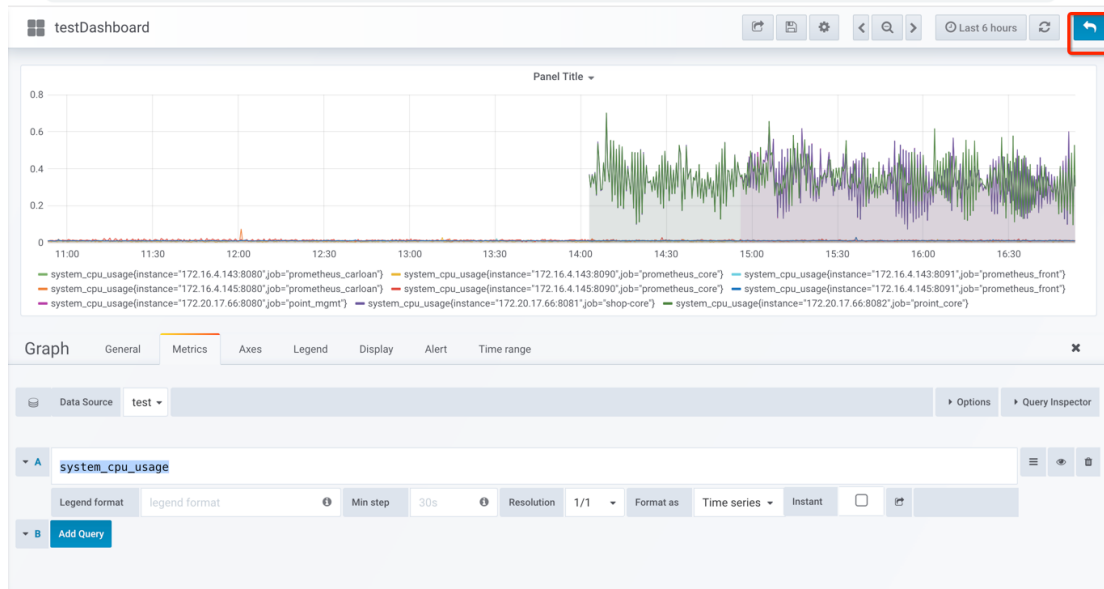
一般需要注意的是两个地方： general metrics
点击 general 可配置 panel 的 name 等相关信息：



点击 metrics 选择配置好的数据源 填写 prometheus 查询语句（实例中是 system_cpu_usage,CPU 使用率。若需要监控该项, 则需要是在被监控的宿主主机上安装 node_exporter）



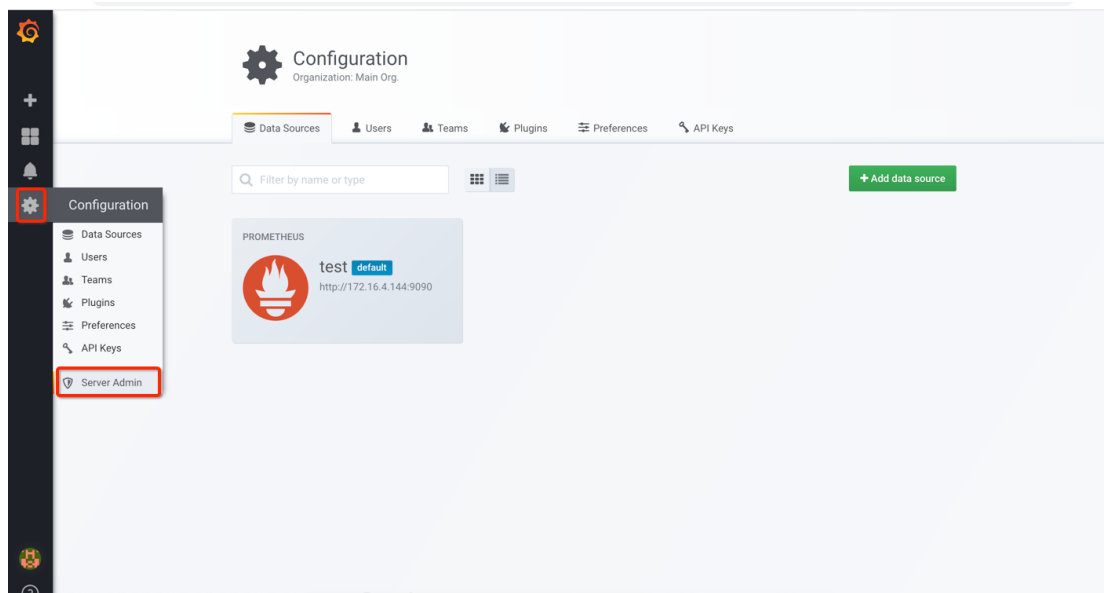
返回：

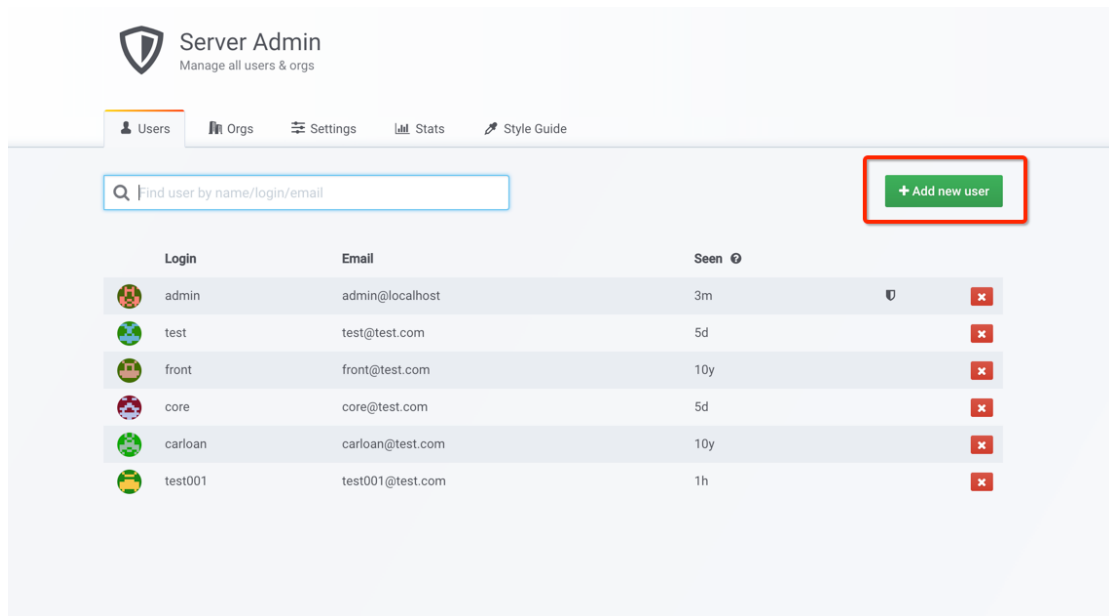


这里可以修改展示数据的时间间隔、刷新。保存即可。

添加用户

以管理员账号登录：以此点击





填写相关信息：

Users Orgs Settings Stats Style Guide

Add new user

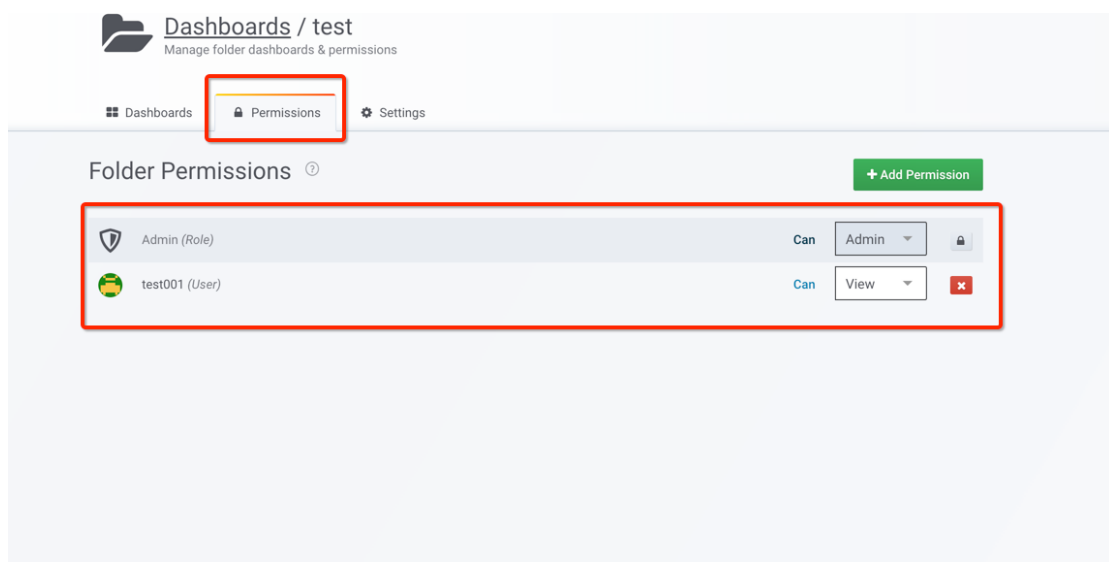
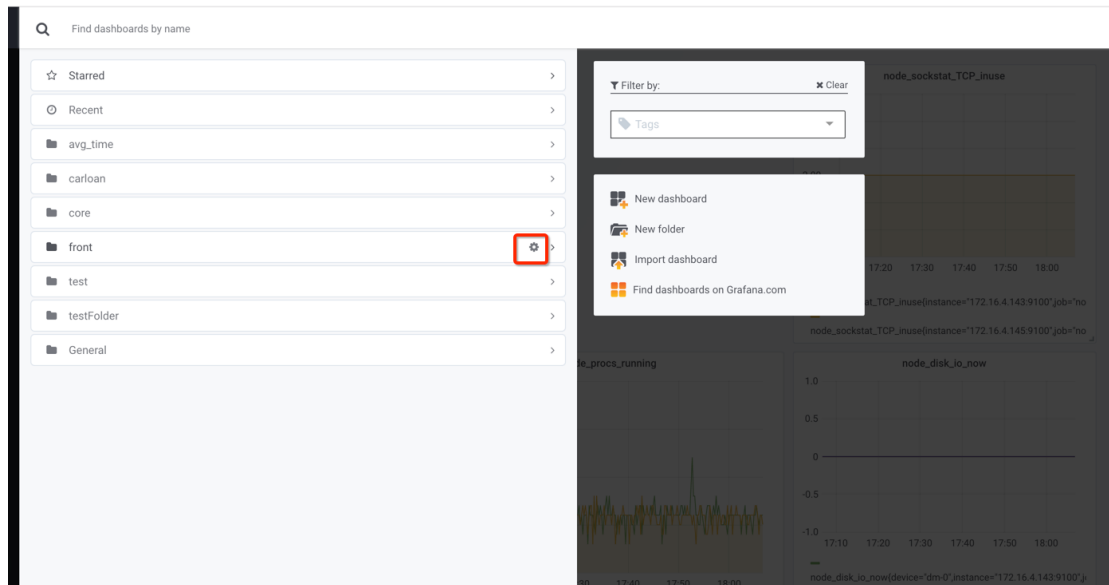
Name	
Email	
Username	
Password	

Create

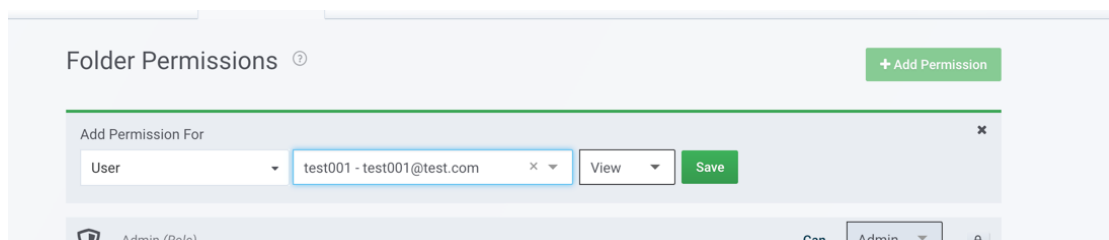
添加用户完成。

分配权限

以 folder 为单位，为不同用户添加不同权限
点击文件夹后的设置按钮：



删除无关用户 点击 add permission 选择新创建的用户 save 即可。



可配置的部分监控项目

上一步的实例中，以 CPU 使用率做事例，配置监控。配置其余监控项步骤

几乎相同。区别只在于 prometheus 的查询语句不同，下面列举常用的监控项需要用到的 prometheus 查询语句。

应用相关：

```
rate(http_server_requests_seconds_sum{instance="ip:port"}[1m])/rate(http_server_requests_seconds_count{instance="ip:port"}[1m])
```

将 ip port 替换为当前应用对应的。统计所有 http 接口一分钟内的平均耗时。

```
rate(http_server_requests_seconds_sum{instance="ip:port",uri="/contract/create"}[1m])/rate(http_server_requests_seconds_count{instance="ip:port",uri="/contract/create"}[1m])
```

将 ip port 替换为当前应用对应的。统计 /contract/create 接口一分钟内的平均耗时。

```
http_server_requests_seconds_max{instance="ip:port"}
```

将 ip port 替换为当前应用对应的。统计当前应用 http 接口最大耗时。

```
http_server_requests_seconds_count
```

http 请求次数，默认监控所有请求；

```
http_server_requests_seconds_max
```

http 请求最大耗时，默认监控所有请求；

```
http_server_requests_seconds_sum
```

http 请求总耗时，默认监控所有请求；

```
tomcat_threads_current
```

当前 tomcat 使用线程数

```
tomcat_threads_config_max
```

tomcat 配置的最大线程数

tomcat_threads_busy

tomcat busy 状态线程数

Java 相关

jvm_classes_loaded

JVM 加载的类的个数；

jvm_gc_pause_seconds_sum

GC 总耗时；

jvm_gc_pause_seconds_max

GC 最大耗时；

jvm_memory_committed_bytes

提供给 JVM 的内存，包括(heap: "PS Eden Space","PS Survivor Space","PS Old Gen";nonheap: "Code Cache","Metaspace","Compressed Class Space");

jvm_memory_used_bytes

JVM 各个区域已使用的内存;

jvm_threads_daemon

JVM 守护线程数量;

jvm_threads_peak

JVM 线程峰值;

jvm_threads_live

JVM 存活的线程;

操作系统相关

node_memory_Active_bytes

当前机器活跃的内存大小

node_memory_MemFree_bytes{instance="ip:port"}

当前机器空闲内存大小

node_procs_running

运行中的 cpu 核心数量;

node_sockstat_TCP_inuse

正在使用中的 TCP 连接数量

process_cpu_usage

当前 JVM 的 CPU 使用率

system_cpu_usage

整个系统的 CPU 使用率